Math 421/510: Homework 6

Dominic Klukas

SN: 64348378

1 Introduction

2 Background

2.1 Multi Agent Reinforcement Learning

In this paper, we are developing Assume Guarantee contracts in a multi-agent learning context. In particular, we will be using reinforcement learning, which is grounded in Markov Decision Processes (MDPs). An MDP concists of a tuple $\mathcal{M} = (S, A, P, R, \gamma, \mu)$, where S is a state space, A is a set of actions, $P : S \times A \times S \to [0, 1]$ is a transition function, $R : S \times A \to \mathbb{R}$ is a reward function, $\gamma \in (0, 1]$ is a discount factor, and $\mu : S \to [0, 1]$ is an initial state distribution.

MDPs are extended to the co-operative multi-agent setting by writing the state space of the team of agents as $S = S_1 \times \ldots \times S_n$, the product of the individual state spaces. Likewise, at each state the action space becomes the product of the individual action spaces of the agents: $A = A_1 \times \ldots \times A_n$, with the transition function also tracking interactions between the dynamics of the individual agents. Within the scope of this paper, we will assume the special case that the dynamics of the individual agents are independent suffer no interaction effects, so that the individual transition functions of the individual agents completely determines the team transition functions is: $P_{team}(S', A, S) = \prod_{i=1}^{n} P(S'_i, A_i, S_i)$.

2.2 Assume/Guarantee Contracts

Contract Theory is a for that prevents integration failures in systems design by verifiably expressing so called assumptions and guarantees for each component in order for them to function properly within their system. The assumptions are the set of requirements a component needs in order to function properly. The guarantees express the desired behavior of the component, given the assumptions hold. A contract expresses the set of assumptions for and guarantees required of a particular part of a system, which a component then implements.

- What are A/G contracts and why are they helpful in this case.
- How do they work and what do they do?
- Components
- Contracts
- I have not introduced automata yet

3 Methodology

3.1 Reward machines as composition of subtasks

To answer our question, we need to show that an arbitrary reward machine can be written as a composition of subtasks that require the minimal amount of information for them to complete the task.

Theorem 1. Let $\mathcal{R} = \langle U, u_I, \Sigma, \delta, \sigma \rangle$ be a reward machine, with a final sink goal state. Let m be the number of transitions in the reward machine. Then, there exist m receptive i/o-reward machines $\mathcal{R}_1, \cdots, \mathcal{R}_m$ with a maximum of 4 states and one output action each such that $\mathcal{R}$ is isomorphic to $\mathcal{R}_1 \times \cdots \times \mathcal{R}_m$. Furthermore, this decomposition can be calculated in $O(f(n, m))$ time, where n is the size of the state space of the reward machine.

Proof. Denote the members of the set δ as $d_1, \ldots, d_m$. Let $\mathcal{R}'$ be the reward machine defined such that $\Sigma = \{d_1, \ldots, d_m\}$, so the actions are the transitions. This is equivalent to every action being used for a single edge in the graph of the reward machine. For each transtion d_i, we construct the i/o-reward machine R_i as follows. Let $\Sigma^i_{\text{in}} = \delta \setminus \{d_i\}$, and $\Sigma^i_{\text{out}} = \{d_i\}$. Let $U_i = \{u^i_{\text{start}}, u^i_{\text{active}}, u^i_{\text{loop}}, u^i_{\text{end}}\}$.

Next, we construct the transitions between these states, given by the transition relation δ_i. Now, for each state $u \in U$, let $r(u)$ denote the set of elements such that for $v \in r(u)$, there exists some path $v \to u$, and let $s(u)$ denote the set of elements such that for $v \in s(u)$, there exists some path $u \to v$, where both sets exclude u. Now, let u_i be the source state of the transition d_i. Then, leth

$$\begin{aligned} U^i_{\text{active}} &= \{u_i\} \\ U^i_{\text{loop}} &= r(u_i) \cap s(u_i) \\ U^i_{\text{start}} &= r(u_i) \setminus s(u_i) \\ U^i_{\text{end}} &= U \setminus (\{u_i\} \cup r(u_i)). \end{aligned}$$

Now, for the states $u^i_j \in U_i$, add the transitions

$$u^i_j \to_{d_l} u^i_k \Leftrightarrow \exists u \in U^i_j, v \in U^i_k \text{ s.t. } d_l = (u, \alpha, v) \text{ for some } \alpha \in \Sigma.$$

This will also add self loops, if u and v are in the same partition. Also, since there only exists one pair, $u, v \in U$ such that $u \to_{d_l} v$, it follows that only one transition gets added with action $d_l \in \Sigma$. Finally, let the initial state u^i_I be such that $I = j$, where U^i_j is the partition such that $u_I \in U^i_j$.

For the reward, consider that d_i's starting state is u_i, which means that the transition d_i goes from u_i to some $v \in U^i_{\text{loop}}$ or $v \in U^i_{\text{end}}$. Thus, d_i is an action in $\mathcal{R}_i$, for some transition going from u^i_{active} to either u^i_{loop} or u^i_{end}, which I will call u^i_j. In $\mathcal{R}$, it is a transition between some u and v. Then, way, let $\sigma_i(u^i_{\text{start}}, u^i_j) = \sigma(u, v)$. For all other u^i, v^i, let $\sigma_i(u^i, v^i) = 0$. This completes the construction of $\mathcal{R}_i$.

Now, we prove that $\mathcal{R}$ and $\mathcal{M} = \mathcal{R}_1 \times \ldots \times \mathcal{R}_m$ are isomorphic as graphs, after the actions are ambiguated. We do so by explicilty constructing a bijection between $\mathcal{R}$ and the reachable states of $\mathcal{M}$, and showing that it is an isomorphism of automata.

Let $\rho^i : U \to U_i$ be the map defined such that for $s \in U$, $\rho^i(s) = u^i_j \Leftrightarrow s \in U^i_j$. That is, this map identifies states in U with their partition. Let $f = (\rho_1, \ldots, \rho_n)$.

We show that f is a bijection between the states of $\mathcal{R}'$ and the reachable states of $\mathcal{M}$. To do so, we have to show that the model is injective, and that any state not in $f(U)$ is not reachable.

Suppose $f(s_1) = f(s_2)$. Suppose that there is some transition going from s_1. If s_1 is the source state of some transition, d_k. Then, this implies that $f(s_1) = (u^1_{j_1}, \ldots, u^k_{\text{active}}, \ldots, u^m_{j_m}) = f(s_2)$. Then, $\rho_k(s_1) = \rho_k(s_2) = u^k_{\text{active}}$, and then $s_1, s_2 \in U^k_{\text{active}}$. However, this implies that $s_1 = s_2$, since $|U^i_{\text{active}}| = 1$ for every i. Now, suppose that s_1 is not a source of some transition. This implies that s_1 is the sink state. Now, the sink state is the only state which is not the source of any transition, so it is the only state such that $\rho^k(s_1) \neq u^k_{\text{active}}$. Therefore, $f(s_1) = f(s_2)$, since the output completely determines the input.

Next, we show that this preserves transitions between the automata, so that this is an injective homomorphism. Suppose that $s_1 \to_{d_k} s_2$. We need to show that this implies $f(s_1) \to_{d_k} f(s_2)$. It suffices to show that, for all i, $f^i(s_1) \to d_k f^i(s_2)$. Denote $f^i(s_1) = u^i_{j_1}$, and $f^i(s_2) = u^i_{j_2}$. In particular, from the definition of f, we have that $s_1 \in U^i_{j_1}$, and $s_2 \in U^i_{j_2}$. But then, from the definition of $\mathcal{R}_i$, $s_1 \in U^i_{j_1}$ and $s_2 \in U^i_{j_2}$ with $s_1 \to_{d_k} s_2$ is precisely the existence condition that implies $u^i_{j_1} \to_{d_k} u^i_{j_2}$ for all $1 \leq i \leq m$, so by the definition of composition of automata, we also have $f(s_1) \to_{d_k} f(s_2)$.

Now, suppose that $f(s_1) \to_{d_k} f(s_2)$. Then, $f^i(s_1) \to_{d_k} f^i(s_2)$ for each $1 \leq i \leq m$ by the definition of composition of automata. In particular, $f^k(s_1) \to_{d_k} f^k(s_2)$, which implies that $s_1 \in U^k_{\text{active}}$. Therefore, s_1 is the source state for the transition d_k, so there exists some s such that $s_1 \to_{d_k} s$. But then, $f(s_1) \to_{d_k} f(s)$. However, we have noted that the action d_k only applies on one transition in each $\mathcal{R}_i$, so the end states of the transition $f^i(s_1) \to_{d_k}$ are unique, so that $f(s) = f(s_2)$. Since f is injective, it follows that $s = s_2$, so that $f(s_1) \to_{d_k} f(s_2)$.

Finally, we show that f is surjective onto the reachable states. Let $t \in \mathcal{M}$ be a reachable state. Then, there exists some sequence of actions $d_{j_1}, d_{j_2}, \ldots, d_{j_n}$ such that $t_I \to_{d_{j_1}} t_1 \ldots \to_{d_{j_n}} t$. We prove by induction on n that there exists a state $u \in U$ such that $f(u) = t$. Suppose $n = 0$. Then, $t = t_I$, and $f(u_I) = t_I$ satisfies our requirement. Next, suppose that for all sequences of actions length $n-1$, t_{n-1} is reachable. Then, there exists some u_{n-1} such that $f(u_{n-1}) = t_{n-1}$. In particular, we have that $f(u_{n-1}) \to_{d_{j_n}} t$. However, since $u_{n-1} \in U^{j_n}_{\text{active}}$, it follows that there exists some u such that $u_{n-1} \to_{d_{j_n}} u$. Then, we have that $f(u_{n-1}) \to_{d_{j_n}} f(u)$, since f preserves transitions between automata. However, each action appears in exactly one transition in the automata $\mathcal{M}$, so we have that $f(u) = t$, as desired. Therefore, f is surjective under the reachable states. $\square$